

ER–Schema Consistency Checker

Anh Nguyen, Jia Yee Wong, Jun Kim

Abstract

Ensuring consistency between an Entity-Relationship (ER) diagram and its corresponding relational schema is a foundational requirement in database design. Misalignment between conceptual and logical levels can introduce significant errors, including missing tables, incorrect definitions of primary keys, or improperly implemented relationships. This project introduces an ER-Schema Consistency Checker, a lightweight tool that automates the verification of ER-to-Schema alignment by evaluating entity presence, attribute coverage, key constraints, and relationship translation accuracy.

Recent research highlights the growing importance of automated reasoning in data modeling workflows. [1] Who evaluated large language models' capability to interpret and generate descriptions from ER schema-based conceptual data models, most existing approaches focus on documentation or translation rather than systematic consistency verification. Similarly, [2] examined the computational complexity of verifying transactional consistency, showing that correctness checking, even in well-defined formal systems, can be nontrivial and computationally demanding. These works collectively underscore the need for practical tools that improve reliability in database design.

Building on these motivations, our system employs both rule-based structural validation and semantic checks to detect common inconsistencies between ER diagrams and relational schemas. Experiments using multiple toy datasets, including intentionally inconsistent designs, show that the tool successfully identifies missing tables, incorrect foreign-key mappings, and errors in translating cardinalities. Overall, the ER-Schema Consistency Checker offers an accessible and extensible approach for supporting conceptual-to-logical model verification in educational and early development settings.

I. INTRODUCTION

Conceptual database design plays a critical role in ensuring that information systems are structured, maintainable and logically sound. At the center of this design process is the Entity-Relationship (ER) model, which provides an intuitive, high-level depiction of how data elements are relate to one another in a domain. Once an ER model is created, it must be translated into a relational schema, a set of tables, attributes, primary keys and foreign keys that can be implemented in an actual database system. Although this transformation follows well-established rule, the process is still prone to errors, especially when performed manually or under time constraints. Common issues include incomplete table translations, incorrect representation of cardinalities, improper handling of weak entities, and naming inconsistencies that make the schema difficult to interpret or verify.

Because ER-to relational schema translation is one of the earliest stages of database development, errors introduced at this point can propagate throughout the entire system. Even minor inconsistencies can lead to data redundancy, lost information, or incorrect relationships that compromise the quality of queries and constraints. As database systems grow in size and complexity, the need of automated support during early design stages becomes increasingly important. Recent research reinforces this trend. [1] showed that modern language models are capable of understanding ER schemas and generating meaningful descriptions from them, suggesting that conceptual models can be processed and analyzed automatically. In another domain of correctness analysis, [2] demonstrated that verifying consistency in transactional systems can involve significant computational challenges, signaling that correctness checking is a nontrivial problem even for formally defined models.

Despite these advances, tools that directly assess the consistency between an ER diagram and its relational schemas automatically, providing tutorials, or checking constraints; yet few offer a structured way to verify whether a student-generated or manually designed schema truly reflects the original conceptual model. To address this gap, this project introduces the ER-Schema Consistency Checker, a tool build to identify mismatches between conceptual and logical designs. By offering structured feedback on missing entities, misaligned relationships, attribute discrepancies, and key-constraint errors, the tool aims to support more reliable database design practices and assist students and developers in producing accurate, high-quality schemas.

II. METHODS

The goal of this project is to automatically check whether a relational schema correctly implements a conceptual Entity-Relationship (ER) diagram. Each input consists of two parts: an ER diagram and a relational schema. The ER diagram describes entities, their attributes and primary keys, and binary relationships between entities, annotated with cardinalates such as 1–N or M=] ?N. The relational schema describes the implementation of this design in terms of tables, their columns and primary keys, and foreign key constraints. The method determines whether the schema is consistent with the ER diagram, producing

either a binary decision (YES/NO) for a single ER-schema pair or evaluation scores for accuracy, precision, and recall over a dataset of labeled examples.

Both of the ER diagram and the relational schema are represented in JSON to make them easy to parse programmatically. The ER JSON file `er.json` contains a dictionary of entities, where each entity is associated with a list of attribute names and a primary key attribute. It also contains a list of relationships, where each relationship has a name, exactly two participating entities, and a cardinality string that is either 1–N or M–N. The relational schema JSON file `schema.json` contains a dictionary of tables, where each table stores a list of its columns, a list of columns forming its primary key, and a list of foreign key constraints. Each foreign key constraint includes the local foreign key columns and a references block that specifies the referenced table and the referenced primary key columns.

Both validation methods share a common preprocessing stage. First, the script `er_schema_validator.py` reads the ER and schema JSON files into Python dictionaries. It then performs name normalization to make entity–table matching more robust, converting names to lowercase and removing all non-alphanumeric characters. This allows the system to match names such as "Student" with "student" and "Order Item" with "order_item" in a consistent way. Using these normalized names, the code builds a mapping from each ER entity to a schema table. It first tries an exact match between normalized entity names and normalized table names. If that fails, it applies a simple singular/plural heuristic by adding or removing a trailing "s" and checking again. The resulting entity–table mapping is then reused by both methods.

The first method, the simple heuristic method, is a rule-based approach that combines name matching, attribute coverage, and explicit foreign key checks. For each entity in the ER diagram, the method checks whether a corresponding table has been found in the schema. It then verifies that the entity's primary key appears as a column in that table and is included in the table's primary key definition. The method also considers attribute coverage by computing the fraction of the entity's attributes that actually appear as columns in the mapped table. If this fraction falls below a threshold of 0.6, the mapping is flagged as suspicious. Additionally, the simple method enforces relationship rules derived from the ER diagram. For a 1–N relationship between entities A and B, it checks that the table corresponding to B has a foreign key pointing to the table corresponding to A. For an M–N relationship between entities A and B, it searches for a distinct join table that has foreign keys referencing both A's table and B's table. If any of these checks fail because tables are missing, keys are missing, attribute coverage is too low, or the required foreign keys or join table are absent, the method classifies the ER-schema pair as invalid (NO). If none of the checks fail, the pair is classified as valid (YES).

The second method is a graph-based schema matching approach that focuses more on the structure of the schema rather than attribute coverage. It models the relational schema as a directed graph, where nodes represent tables and directed edges represent foreign key relationships from child tables to the referenced parent tables. Using the same entity table mapping as the first method, it verifies that each entity's primary key is present as a column and is part of the primary key in the corresponding table. Unlike the simple heuristic method, it does not require a particular fraction of attributes to appear, making it more tolerant of schemas where only key attributes are kept or some non-key attributes are omitted. Once the graph is built, it checks whether it contains the expected structural patterns implied by each relationship in the ER diagram. For a 1–N relationship from entity A to entity B, it looks for a directed edge in the graph from the N-side table (B's table) to the 1-side table (A's table), reflecting the expected foreign key direction. For an M–N relationship between A and B, it searches for a third table J such that there is an edge from J to A's table and another edge from J to B's table. This pattern corresponds to the presence of a join table referencing both participating entities. If all the required nodes and edges exist, the ER-schema pair is classified as valid, otherwise, it is classified as invalid.

III. DATA

This project uses a small synthetic dataset of ER-schema pairs to test and demonstrate the two validation methods. Each pair contains one ER diagram and one candidate relational schema, where the schema either correctly implements the ER design or has a deliberate inconsistency.

The dataset is stored in a folder called `toy_dataset`. Each test case is a subfolder and contains three files: `er.json` (the ER description), `schema.json` (the corresponding relational schema), and `label.json` (the ground-truth consistency label). The label file stores whether that ER-schema pair should be considered valid or invalid using a Boolean field `"is_valid"` set to either `true` or `false`. The validation code only assumes this generic JSON structure, so the same program can be applied to other datasets that follow the same format.

The toy dataset contains four cases designed to cover both 1–N and M–N relationships as well as typical sources of inconsistency. The first case (`case1_valid_enrolls`) models a many-to-many relationship between Student and Course with a correct join table implementing the Enrolls relationship, and this case is labeled as valid. The second case (`case2_invalid_missing_join`)

uses the same ER diagram but omits the join table in the schema, so the many-to-many relationship is not implemented properly, hence this case is labeled as invalid. The third case (*case3_dep_emp_valid*) represents a 1-N relationship between Department and Employee, where the schema includes a foreign key from the employee table to the department table, therefore, this case is labeled as valid. The fourth case (*case4_dep_emp_sparse*) uses the same ER diagram as the third case but a more sparse schema in which some non-key attributes are omitted while the primary and foreign keys remain correct, hence this case is still labeled as valid.

Overall, the dataset contains three valid schemas and one invalid schema. Although small, it is designed to highlight the different behavior of the two methods: the simple heuristic method is sensitive to missing attributes as well as missing keys, whereas the graph-based method focuses on the structural correctness of primary and foreign keys.

IV. RESULTS

In evaluating our ER-Schema Consistency Checker, we applied both the Simple Heuristics method and the Graph-based method to four toy dataset cases. Each case included a conceptual ER model, a relational schema, and a ground-truth label showing whether the two were consistent. As we analyzed the results, we organized the predictions and overall performance into Tables 1 and 2, which helped us see more clearly how the two approaches compared across different scenarios.

In evaluating our ER-Schema Consistency Checker, we applied both the Simple Heuristics method and the Graph-based method to four toy dataset cases. As we analyzed the results, we organized the predictions and overall performance into Tables 1 and 2, which helped us see more clearly how the two approaches compared across different scenarios.

Case	Truth	Simple Method	Graph Method
case1_valid_enrolls	TRUE	TRUE	TRUE
case2_invalid_missing_join	FALSE	FALSE	FALSE
case3_dep_emp_valid	TRUE	TRUE	TRUE
case4_dep_emp_sparse	TRUE	FALSE	TRUE

TABLE I

The Simple Heuristics method misclassified case4, while the Graph-based method correctly classified all four cases.

Metric	Simple Method	Graph-based Method
n (cases)	4	4
Accuracy	0.75	1
Precision	1	1
Recall	0.67	1



Fig. 1. Accuracy, precision, and recall comparison between the Simple Heuristics method and the Graph-based method.

Fig. 2. Figure 2. Case-by-case prediction visualization comparing truth labels with predictions from both validation methods.

This figure highlights the performance difference between the Simple Heuristics Method and the Graph-Based approach. The Graph-Based method reaches perfect accuracy, precision, and recall, showing that it aligns entirely with the ground truth across all four cases. In comparison, the Simple Method falls short in recall because of one false negative in case 4. Once visualized in the chart, the contrast becomes easy to see, especially in how the Graph-Based approach captures structural relationships more effectively.

The second figure builds on this by presenting the truth values and predicted outputs side by side. Here, it becomes clear that the Simple Method misclassifies case 4, while the Graph-Based Method remains accurate in every case. This comparison illustrates how the two methods respond differently when the schema is incomplete or structurally ambiguous. The Simple

Method works well when attribute names and foreign keys are clearly defined but tends to struggle when those indicators are vague. Meanwhile, the Graph-Based Method continues to make correct predictions even when the schema lacks detail, reinforcing its overall reliability.

The quantitative results further emphasize the gap between the two approaches. The Simple Method reached 75 percent accuracy and perfect precision, but its recall dropped because of the false negative. This suggests that the method sometimes misses valid relationships when structural information is not directly represented. The Graph-Based Method, on the other hand, achieved perfect scores across all metrics, confirming that it not only avoids false positives but also captures all valid cases correctly.

Another important observation appears when looking at how each method behaves across different levels of schema complexity. In simpler scenarios—where entity structures, attributes, and foreign keys line up neatly—the Simple Method performs almost as well as the Graph-Based approach. However, once the schema becomes more abstract or contains fewer explicit attribute matches, the Simple Method becomes less dependable. The misclassification in case 4 shows how even a small lack of attribute overlap can lead to incorrect results. In contrast, the Graph-Based Method remains unaffected by these variations because it relies on structural relationships within the schema graph rather than surface-level features.

Overall, the results show that the Graph-Based Method is noticeably more reliable for ER-schema consistency validation. The Simple Method still works well when the schema is straightforward and fully detailed, but its accuracy decreases when the structure becomes sparse or unclear. The Graph-Based Method, by identifying deeper patterns within the schema graph, achieves perfect classification across all four cases. These findings highlight that structural analysis provides a stronger foundation for validation and is better suited for handling schemas with varying levels of complexity.

V. DISCUSSION

The experimental results offer helpful insight into the strengths and limitations of the two validation methods. The Simple Heuristics Method was able to classify most cases correctly, but its mistake on case 4 reveals a meaningful weakness. Because this method relies heavily on attribute overlap and explicit foreign-key matches, it works best when the schema is complete and the relationships are clearly represented. When the schema becomes sparse, abbreviated, or only implies structure rather than explicitly modeling it, the Simple Method is more likely to misinterpret the relationship. This makes it less dependable for datasets that are not perfectly documented or that come from more complex design environments.

In contrast, the Graph-Based Method consistently achieved perfect performance across all four test cases. By modeling the relational schema as a graph, it can recognize deeper structural patterns—such as 1-N or M-N relationships—without depending solely on attribute names. This focus on structural connectivity allows the method to generalize well, even when attributes are missing, inconsistently named, or incomplete. Because real-world schemas often vary in quality and are built by multiple developers, this level of robustness becomes especially important.

Another important observation is the difference in the types of errors each method tends to produce. The Simple Method's error in case 4 was a false negative, meaning it failed to recognize a valid relationship. In practice, false negatives can create unnecessary design concerns or lead to incorrect assumptions about a mismatch between the conceptual model and the schema. The Graph-Based Method avoided these issues entirely, showing that it is better suited for situations where structural accuracy is essential.

Overall, the findings suggest that the Graph-Based Method is the more reliable and scalable option for ER-schema consistency checking. Even so, the Simple Method still serves a purpose as a quick, lightweight first pass when speed or limited computation matters. Looking ahead, combining both approaches—using the Simple Method for fast initial screening and applying the Graph-Based Method when deeper analysis is needed—may offer a balanced way to improve both efficiency and accuracy.

VI. CONCLUSION

This project set out to address a persistent challenge in database design: ensuring that a relational schema faithfully implements the structure defined in an Entity-Relationship (ER) diagram. The ER-Schema Consistency Checker developed for this work offers a practical solution by automating this verification process through two complementary approaches, which are Simple Heuristics Method and a Graph-Based Method. By applying these methods to a curated set of toy datasets, we were able to observe clear differences in how each approach interprets conceptual-logical alignment.

The Simple Heuristics Method demonstrated strong performance in cases where schemas were complete and explicit, correctly identifying most inconsistencies in attribute coverage, key definitions, and foreign-key placement. However, its misclassifications of the sparse schema in case 4 revealed its sensitivity to missing or incomplete attributes. This limitation underscores the challenges identified in existing research, such as the difficulty of analyzing conceptual models when representation is incomplete, as highlighted by [2]’s work on consistency checking complexity. In contrast, the Graph-Based Method proved more robust, achieving perfect accuracy, precision, and recall across all test cases. By modeling the schema as a structural graph, this method aligned more closely with the capabilities demonstrated by [1], who showed how automated approaches can interpret relationships in conceptual data models even when traditional cues are limited.

Overall, the results show that structural analysis provides a more reliable basis for verifying ER-to-Schema consistency, especially when schemas vary in completeness or naming precision. While the Simple Method offers a lightweight option for quick checks, the Graph-Based Method is better suited for handling real-world variability and imperfect designs. Future work could focus on expanding the dataset, applying the checker to larger or real student submissions, integrating semantic similarity models, or combining both methods into a hybrid system. Such enhancements would help create a more comprehensive and adaptive tool capable of supporting database education, automated design assistance, and early-stage system development.

REFERENCES

- [1] A. Avignone, A. Tierno, A. Fiori, and S. Chiusano, “Exploring large language models’ ability to describe entity-relationship schema-based conceptual data models,” *Information*, vol. 16, no. 5, p. 368, 2025.
- [2] R. Biswas and C. Enea, “On the complexity of checking transactional consistency,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, 2019. [Online]. Available: <https://doi.org/10.1145/3360591>